

MODULE 15 · DOMAIN 2 - DESIGN FOR NEW SOLUTIONS

Containers and Serverless

ECS, EKS, Fargate and Lambda. The question is never simply which one.

DOMAIN 2 of 4 EXAM WEIGHT 29% MODULE 15 of 44

Ask what the **unit of scale** is. A request, a container, or a node. That answers most of the question.

The four options

OPTION	YOU OPERATE	FITS
Lambda	Nothing. Just code	Event-driven, spiky, under 15 minutes
ECS on Fargate	Containers only	Long-running services, no cluster to run
ECS on EC2	Containers and instances	GPU, special instance types, cost at scale
EKS	Kubernetes and its ecosystem	Portability, Helm, operators, existing skills

ON THE EXAM

ECS has no control plane charge. EKS charges hourly per cluster, and costs far more in people.

ECS or EKS?

AMAZON ECS

AWS-native, simpler

- No control plane charge
- Deep IAM and VPC integration
- Small operational surface
- Nothing transfers off AWS

AMAZON EKS

Kubernetes, with its costs

- Helm, operators, CRDs, the ecosystem
- Skills and manifests are portable
- Hourly cluster charge
- You own upgrades and add-ons

THE TIEBREAKER

Existing Kubernetes skills or artefacts means EKS. Anything else, and the simpler operational surface usually wins.

Fargate, and capacity providers

- ◆ **Fargate removes the instances**, and charges a premium per vCPU and GB for that.
- ◆ **Fargate Spot** gives large savings on interruption-tolerant tasks, with two minutes' notice.
- ◆ **Capacity providers** mix Fargate, Fargate Spot and EC2 in one service, with a base and a weight.
- ◆ A new Fargate task takes tens of seconds to start. That is not a Lambda cold start, and it is not instant either.

ON THE EXAM

Fargate is not always cheaper. At steady high utilisation, EC2 with reservations often wins; Fargate wins on operational cost.

EKS specifics worth knowing

- ◆ **Managed node groups** handle provisioning and updates. **Karpenter** provisions right-sized nodes on demand instead.
- ◆ **Fargate profiles** run pods with no nodes at all, selected by namespace and labels.
- ◆ **IRSA**, or EKS Pod Identity, maps a Kubernetes service account to an IAM role. This is the answer for pod-level permissions.
- ◆ **EKS Anywhere** and **EKS Distro** run Kubernetes outside AWS. Both are on the exam's in-scope list.

ON THE EXAM

Pods needing distinct AWS permissions is an IRSA question. Giving the node role everything is the anti-pattern it exists to fix.

Reserved or provisioned concurrency?

RESERVED CONCURRENCY

A ceiling, and a floor

- Caps this function's concurrency
- Also guarantees it that much
- Protects the rest of the account
- No extra charge

PROVISIONED CONCURRENCY

Environments kept warm

- Pre-initialised, ready to serve
- Removes cold starts
- Charged for whether used or not
- Scheduled scaling is common

THE TIEBREAKER

One function starving the others is reserved concurrency. Latency spikes on first invocation is provisioned concurrency.

Lambda's edges

- ◆ **Fifteen minutes maximum.** Ten gigabytes of memory, ten of ephemeral storage, six megabyte synchronous payload.
- ◆ **SnapStart** restores a pre-initialised snapshot. Java first, now Python and .NET too.
- ◆ **Since August 2025 the INIT phase is billed** for every configuration. Cold starts now cost money as well as latency.
- ◆ Functions in a VPC use Hyperplane ENIs, so the old cold start penalty for VPC access is long gone.

ON THE EXAM

Fifteen minutes is the hardest boundary in serverless. A scenario mentioning a longer job has already excluded Lambda.

An honest note on App Runner

- ◆ **App Runner entered maintenance in March 2026** and closed to new customers on 30 April 2026.
- ◆ **It remains on the exam's in-scope service list**, so answer with it where a scenario describes its shape.
- ◆ Its shape: a container or repository in, an HTTPS service out, with no load balancer or scaling to configure.
- ◆ **Amazon ECS Express Mode** is the successor, giving the same simplicity on ordinary ECS and Fargate.

ON THE EXAM

Another examinable service you cannot buy. Teach the answer, know the reality, exactly as with Simple AD in module four.

Reading the scenario

THE SCENARIO SAYS

Existing Helm charts, or multi-cloud

No Kubernetes experience, AWS only

A job runs for several hours

One function is exhausting account concurrency

p99 latency spikes on first request

Each pod needs different AWS permissions

IT IS POINTING AT

EKS

ECS on Fargate

Not Lambda. Fargate or AWS Batch

Reserved concurrency

SnapStart or provisioned concurrency

IRSA or EKS Pod Identity

ON THE EXAM

Check the duration and the trigger first. Those two facts eliminate whole services before you weigh anything else.

Meridian Freight's platform team of four is containerising eleven internal services. They have no Kubernetes experience, run entirely on AWS with no plans to change, and want to spend as little time as possible on platform operations.

A consultant has recommended EKS because it is the industry standard.

What should the solutions architect advise?

- ☐ A EKS with managed node groups, as recommended
- ☐ B ECS on Fargate, with capacity providers for cost control
- ☐ C EKS with Fargate profiles, to avoid managing nodes
- ☐ D Run the eleven services on EC2 with Docker and a load balancer

◆ PAUSE AND DECIDE

What should the solutions architect advise?

A EKS as recommended

Buys a Kubernetes learning curve, upgrade cycle and add-on ecosystem for four people with no Kubernetes experience and no portability requirement.

C EKS with Fargate profiles

The closest miss. It removes the nodes and leaves the entire Kubernetes control surface, which is the actual overhead here.

B ECS on Fargate

No control plane to run, no instances, no Kubernetes to learn, and capacity providers give cost control later.

D EC2 with Docker

Hand-rolls orchestration, health checking and deployment that both managed services provide.

THE SIGNAL

Industry standard is not a requirement. The stated requirements are no Kubernetes skills, AWS only, and minimal operations.

A company acquired a business running forty microservices on Kubernetes on-premises, with a large library of Helm charts, several custom operators, and a team of Kubernetes engineers. They must run in AWS within nine months, and keep the option of returning workloads on-premises later.

The parent company standardises on ECS.

What should the solutions architect recommend?

- ☐ A Rewrite the workloads as ECS task definitions to match the parent standard
- ☐ B Run them on EKS, accepting a documented exception to the ECS standard
- ☐ C Run them on ECS on EC2, since both use containers
- ☐ D Run them as Lambda functions to remove orchestration entirely

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

 Rewrite for ECS
Discards Helm charts, operators and team skills, and spends the nine months rewriting rather than migrating. The portability requirement dies too.
- B

 EKS with an exception
Charts and operators run unchanged, the team's skills apply, and Kubernetes manifests remain portable back on-premises.
- C

 ECS on EC2
Both run containers, and ECS does not run Helm charts or Kubernetes operators. The artefacts do not transfer.
- D

 Lambda
Forty microservices with operators are not functions. This is a rewrite disguised as modernisation.

THE SIGNAL

Pair this with scenario one. Same choice, opposite answer, because existing artefacts and a portability requirement changed everything.

An account runs 40 Lambda functions. During a marketing campaign, one function scaled to thousands of concurrent executions and consumed almost all the account's concurrency. Unrelated functions, including a payment webhook handler, began throttling. The team wants the payment handler always to have capacity.

What should the solutions architect implement?

- ☐ A Request a concurrency quota increase for the account
- ☐ B Set reserved concurrency on the campaign function to cap it, and on the payment handler to guarantee it
- ☐ C Enable provisioned concurrency on the payment handler
- ☐ D Move the payment handler to a separate AWS account

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A Quota increase

Raises the ceiling everyone shares. A bigger pool that one function can still consume entirely is the same failure, later.

C Provisioned concurrency

The classic confusion. It keeps environments warm to remove cold starts, and does nothing to protect a share of account concurrency.

B Reserved concurrency both sides

Reserved concurrency caps the noisy function and simultaneously guarantees the payment handler its own share.

D Separate account

Would work, by adding an account boundary, cross-account plumbing and operational overhead to solve a per-function setting.

THE SIGNAL

Reserved concurrency is a ceiling and a floor at the same time. That dual nature is exactly what this scenario needs.

A customer-facing API runs on Lambda in Java. Median latency is 90 milliseconds, but p99 exceeds 4 seconds because of JVM cold starts. Traffic is steady during the day and near zero overnight.

The team wants predictable latency without paying for idle capacity all night.

What should the solutions architect recommend?

- ☐ A Provisioned concurrency at the daytime peak, kept on permanently
- ☐ B Enable SnapStart, and add scheduled provisioned concurrency for business hours if needed
- ☐ C Increase the function's memory allocation
- ☐ D Rewrite the function in Python

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Permanent provisioned concurrency
Fixes latency and pays for the overnight hours the scenario explicitly rules out. It is right about the mechanism, wrong about the schedule.

C More memory
More memory means more CPU, which shortens init a little. It does not remove a four-second JVM start.

B SnapStart, plus scheduled PC
SnapStart restores a pre-initialised snapshot, which is what JVM cold starts need, and costs nothing while idle.

D Rewrite in Python
Rewriting a working API to change its cold-start profile, when a configuration setting addresses it.

THE SIGNAL

Without paying for idle capacity all night is the constraint. SnapStart costs nothing when the function is not running.

A nightly job reconciles ledgers across systems. It currently runs on Lambda and has begun failing as data volumes grew, timing out at fifteen minutes. It is CPU-bound, single-threaded, and takes about three hours locally.

The team has asked whether the timeout can be raised.

What should the solutions architect advise?

- ☐ A Request a Lambda timeout increase through AWS Support
- ☐ B Split the job into many Lambda invocations coordinated by Step Functions
- ☐ C Move the job to AWS Batch or an ECS scheduled task on Fargate
- ☐ D Increase Lambda memory to 10 GB so it completes faster

◆ PAUSE AND DECIDE

What should the solutions architect advise?

A Raise the timeout

Fifteen minutes is a hard service limit, not an adjustable quota. Support cannot raise it.

C Batch or scheduled Fargate task

No timeout, more CPU available, and a nightly batch job is exactly what these services are for.

B Split with Step Functions

Excellent when work is divisible. This job is single-threaded and sequential, so there is nothing to split it along.

D 10 GB memory

More memory does mean more CPU, and turning three hours into under fifteen minutes needs a twelve-fold speedup. Not plausible.

THE SIGNAL

Single-threaded and sequential is the planted detail. It exists to eliminate the Step Functions answer, which would otherwise be right.

A team runs three internal web tools on AWS App Runner, deployed from container images. They are planning next year's roadmap and ask whether to keep investing in that platform. They want the same simplicity: no load balancer or scaling policy to configure.

What should the solutions architect advise?

- ☐ A Keep App Runner; it is on the AWS service list and works well
- ☐ B App Runner is in maintenance and closed to new customers; plan a move to ECS Express Mode on Fargate
- ☐ C Migrate to EKS for long-term flexibility
- ☐ D Migrate to Lambda behind API Gateway

◆ PAUSE AND DECIDE

What should the solutions architect advise?

- A

Keep investing
It does still work, and a service in maintenance receives no new features and eventually no service. Roadmap questions need that answered honestly.
- B

Plan a move to ECS Express Mode
Preserves the simplicity they value, on ECS and Fargate, which are not going anywhere.
- C

Migrate to EKS
Answers a simplicity requirement with the most operationally demanding option available.
- D

Lambda behind API Gateway
A rewrite from containers to functions, changing the programming model to solve a platform-lifecycle problem.

THE SIGNAL

Note the question is about next year's roadmap. That is a lifecycle question, and lifecycle questions need lifecycle answers.

What to carry out of this module

- ◆ **Ask what the unit of scale is.** A request means Lambda, a process means a container, a node means Kubernetes.
- ◆ **Reserved concurrency is capacity. Provisioned concurrency is latency.** The names mislead; the jobs do not overlap.
- ◆ **Fifteen minutes is hard.** Not a quota, not raisable. Split it, or move it to Batch or Fargate.
- ◆ Existing Helm charts, operators or portability means EKS. Otherwise ECS costs less to operate.

ON THE EXAM

App Runner is examinable and closed to new customers. Teach the answer, know the reality, as with Simple AD in module four.